Fashion Visual Search System

- Tìm sản phẩm từ ảnh outfit
- AI + Full-stack end-to-end
- YOLO, FashionCLIP, pgvector

NHÓM 13



NHÓM 13

Thành viên

Phạm Bá Tiến

B23DCCN822

Triệu Tuấn Anh

B23DCCN053

Nguyễn Minh Vinh

B23DCCN934

Outline

Phần 1:
Bài toán giải quyết

Phần 2:
Công nghệ sử dụng

Phần 3:
Kiến trúc hệ thống

Phần 4:
Giao diện người dùng

Phần 5:
Data Pipeline

Phần 6:
Object Detection Module

Phần 7:
Embedding & Vector Search

Phần 8:
Reranking

Phần 9:
Metric

Phần 10:
Hạn chế

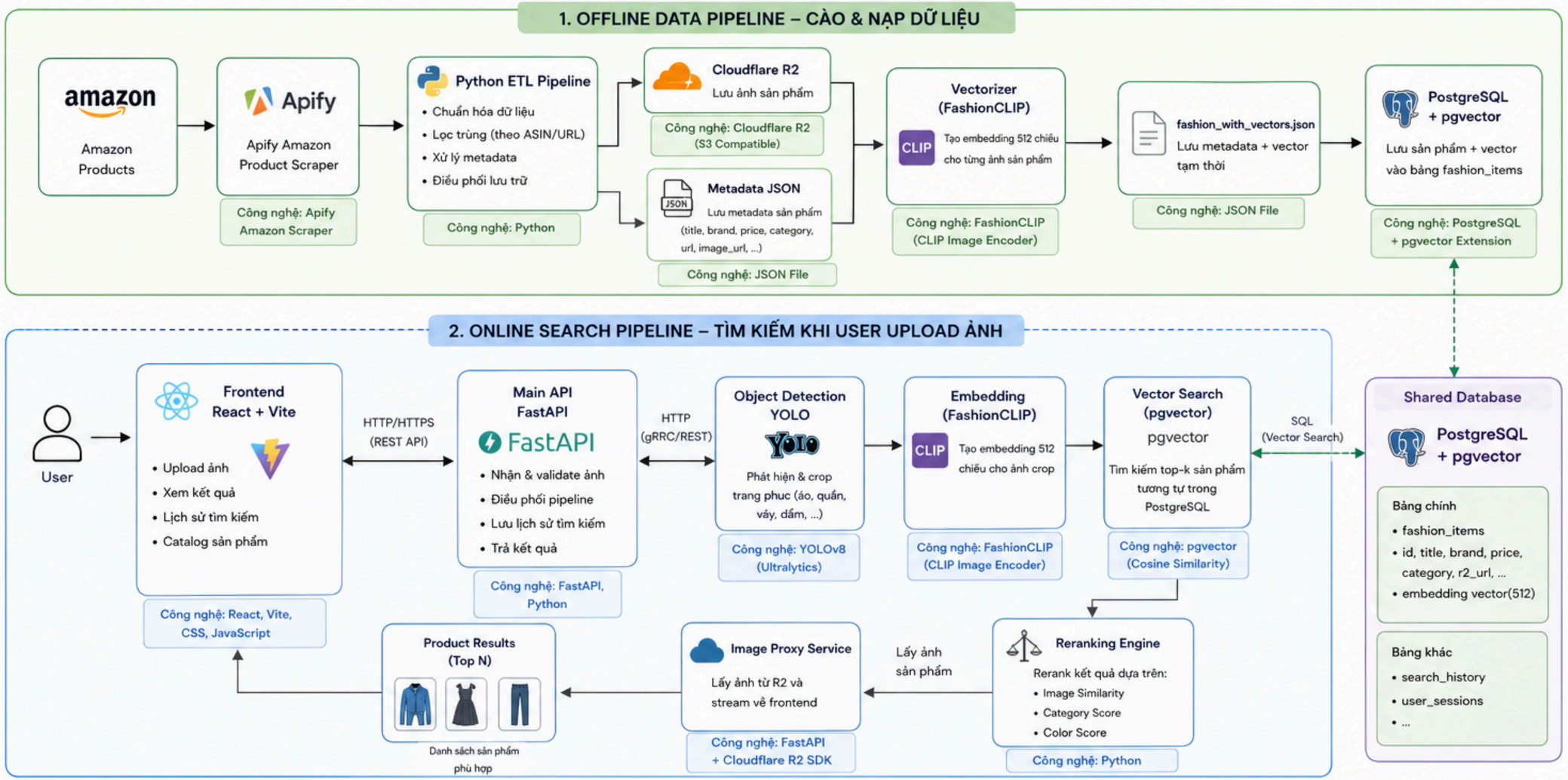
Phần 11:
Kết luận

BÀI TOÁN GIẢI QUYẾT

- Hệ thống crawl sản phẩm trên Amazon
- Người dùng upload ảnh outfit
- Hệ thống nhận diện từng món đồ
- So khớp với catalog đã nạp
- Gợi ý sản phẩm tương tự

KIẾN TRÚC HỆ THỐNG TỔNG THỂ – FASHION SEARCH ENGINE

Kết nối Offline Data Pipeline (Cào & Nạp dữ liệu) và Online Search Pipeline (Tìm kiếm khi upload ảnh)



CÔNG NGHỆ CHÍNH

Frontend	React, Vite, CSS, JavaScript
Backend	FastAPI, Python
AI Models	YOLOv8, FashionCLIP
Database	PostgreSQL, pgvector
Storage	Cloudflare R2 (S3 Compatible)
Scraping	Apify Amazon Scraper
ETL & Processing	Python, JSON
Deployment	Docker, Docker Compose

Deployment	Docker Compose Orchestration
	frontend (5173) | api (8000) | object-detection (8001) | postgres (5433)
Monitoring & Logging	Healthcheck | Logs | Metrics
Environment	.env (R2, DB, API keys, ...)

Data pipeline

Bài toán

- **Mục tiêu:** chuẩn bị catalog sản phẩm trước khi người dùng search.
- **Giải pháp:**
  - Apify dùng để crawl sản phẩm Amazon theo keyword.
  - Ảnh sản phẩm được upload lên Cloudflare R2.
  - FashionCLIP tạo embedding vector 512 chiều.
  - PostgreSQL + pgvector lưu catalog để API search similarity.

Data pipeline

Chi tiết #1

- **Request tới Apify:**
  - Amazon search URL theo keyword
  - maxItems
  - proxyCountry = "US"

- **Response:**
  - asin: Mã định danh sản phẩm trên amazon
  - title
  - brand
  - price
  - thumbnailImage
  - url
  - breadCrumbs

Data pipeline

Chi tiết #2

- **Request tới Apify:**
  - Amazon search URL theo keyword
  - maxItems
  - proxyCountry = "US"

- **Response:**
  - asin: Mã định danh sản phẩm trên amazon
  - title
  - brand
  - price
  - thumbnailImage
  - url
  - breadCrumbs

Data pipeline

Chi tiết #3

- Xử lý dữ liệu:
  - Lọc sản phẩm không hợp lệ: có đủ asin, thumbnailImage
  - Đổi tên field
  - loại sản phẩm trùng bằng ASIN
- Ảnh từ thumbnailImage được tải về và upload lên Cloudflare R2
- Lưu thông tin sản phẩm vào fashion_metadata.json, dưới dạng:

```
{
  "id": "B0ABC123",
  "title": "Men's Black T-Shirt",
  "brand": "ABC Brand",
  "price": 19.99,
  "category": "Men > Clothing > T-Shirts",
  "original_url": "https://www.amazon.com/...",
  "r2_image_url": "s3://fashion-visual-search/images/B0ABC123.jpg"
}
```

→ Dữ liệu đã xử lý sơ bộ

Data pipeline

Chi tiết #4

- Xử lý dữ liệu:
  - Lọc sản phẩm không hợp lệ: có đủ asin, thumbnailImage
  - Đổi tên field
  - loại sản phẩm trùng bằng ASIN
- Ảnh từ thumbnailImage được tải về và upload lên Cloudflare R2
- Lưu thông tin sản phẩm vào fashion_metadata.json, dưới dạng:

```
{
  "id": "B0ABC123",
  "title": "Men's Black T-Shirt",
  "brand": "ABC Brand",
  "price": 19.99,
  "category": "Men > Clothing > T-Shirts",
  "original_url": "https://www.amazon.com/...",
  "r2_image_url": "s3://fashion-visual-search/images/B0ABC123.jpg"
}
```

Data pipeline

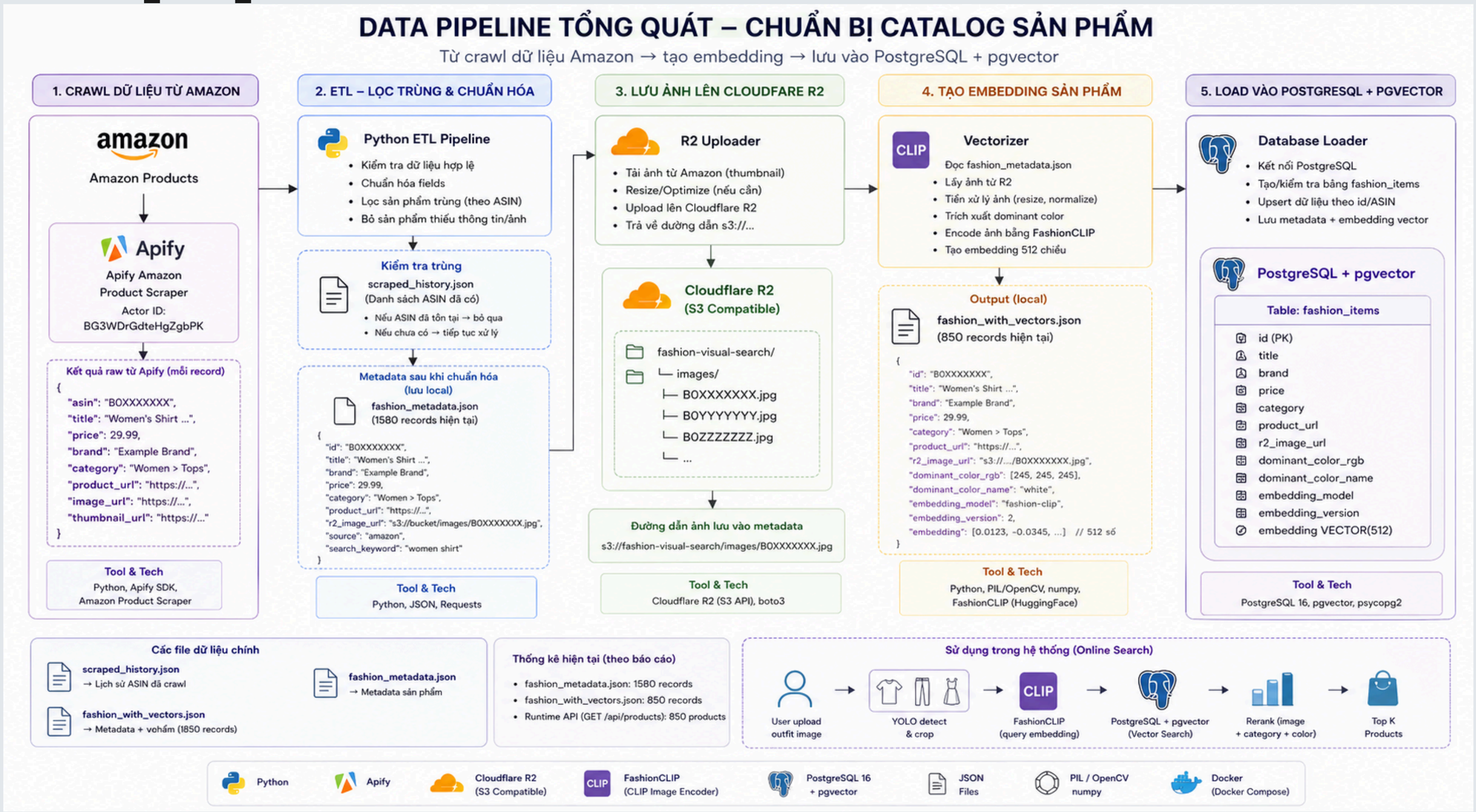
Chi tiết #5

- Đọc fashioon_metadata.json lấy link r2_image_url để gửi request tới cloudflare lấy ảnh
- Tạo embedding bằng model: **patrickjohncyh/fashion-clip**
- Lưu vào PostgreSQL bảng fashion_items:
  - id
  - title
  - brand
  - price
  - category
  - product_url
  - r2_image_url
  - dominant_color_rgb
  - dominant_color_name
  - embedding_model
  - embedding_version
  - embedding VECTOR(512)



Data pipeline

Tổng quan

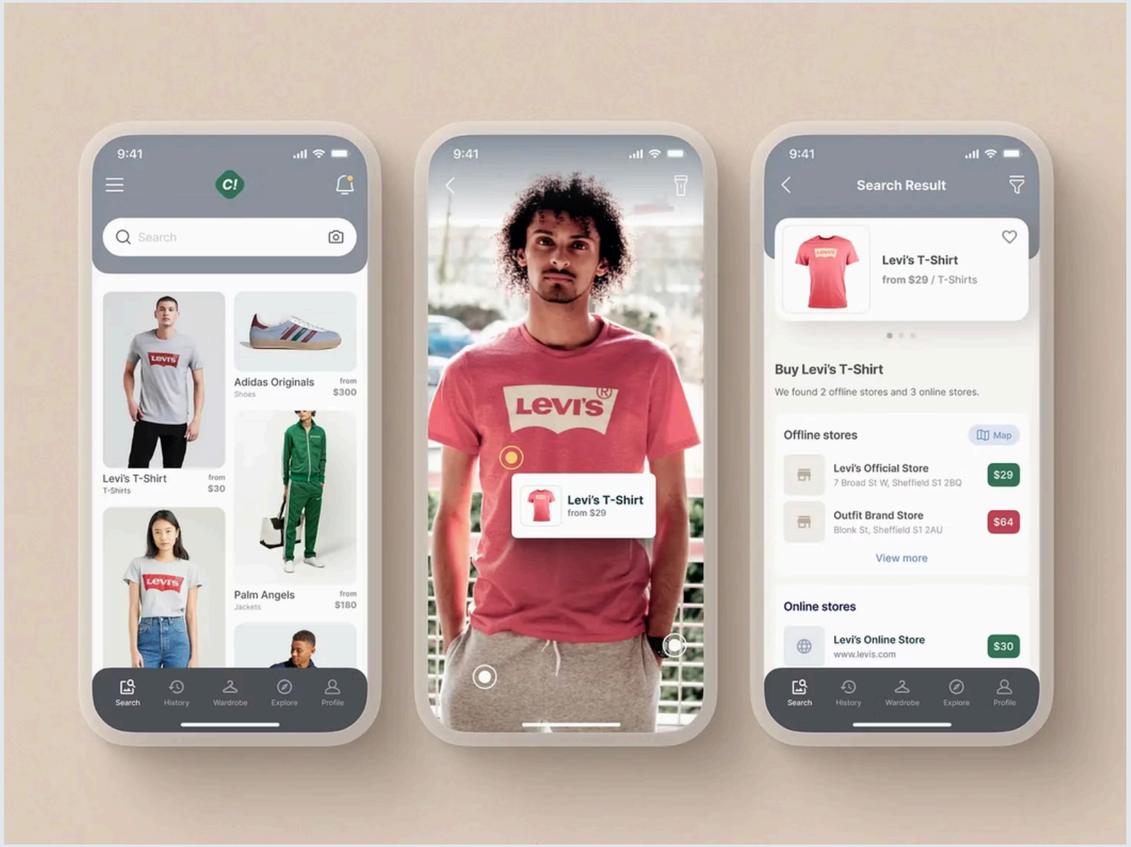


Object detection

Đặt Vấn Đề

Trong hệ thống tìm kiếm trang phục bằng hình ảnh, bước đầu tiên và quan trọng nhất là phải xác định vị trí và phân loại từng món đồ xuất hiện trong ảnh.

- Input: Ảnh một người mặc trang phục
- Output: Danh sách các món đồ được phát hiện, mỗi món gồm:
  - Nhãn lớp trang phục
  - Độ tin cậy dự đoán
  - Tọa độ bounding box
  - Ảnh crop của món đồ
  - Metadata phục vụ bước tìm kiếm tiếp theo



Object detection

DATASET

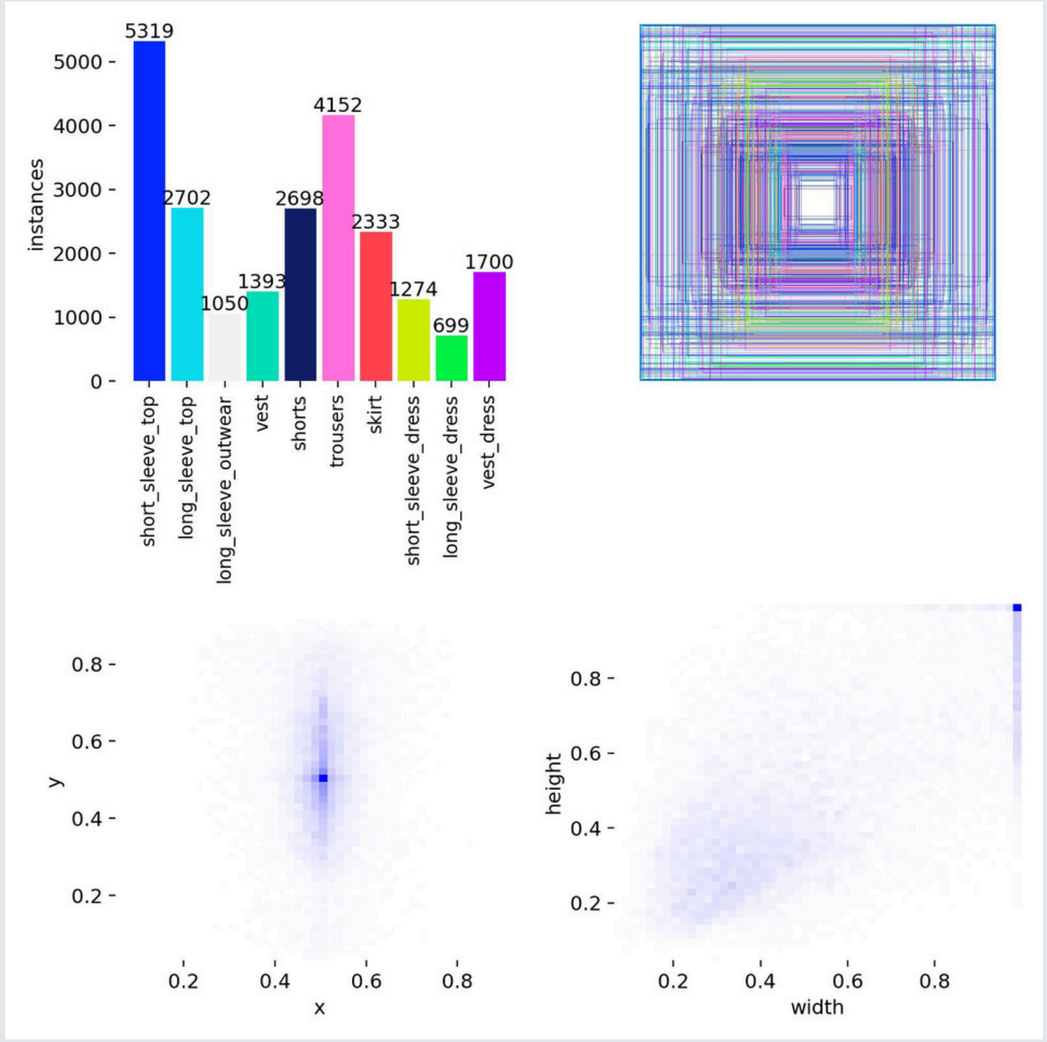
Thông tin	Chi tiết
Tên dataset	DeepFashion2
Số lượng ảnh	~491,000 ảnh (train + val + test)
Số class	10 class trang phục
Format Annotation	JSON (1 file/ảnh), bounding box[x1,y1,x2,y2]

Object detection

DATASET

Dataset hiện tại đang:

- Phân bố các nhãn không đều.
- Quần áo có đủ mọi kích cỡ từ nhỏ đến lớn, nhưng hầu hết đều tuân theo tỷ lệ khung hình chữ nhật đứng (phù hợp với dáng người).
- Các bức ảnh luôn có xu hướng đặt người mẫu mặc trang phục ở chính giữa khung hình..
- Quần áo có thể chiếm 1 phần bức ảnh hoặc toàn bộ khung hình.



Object detection

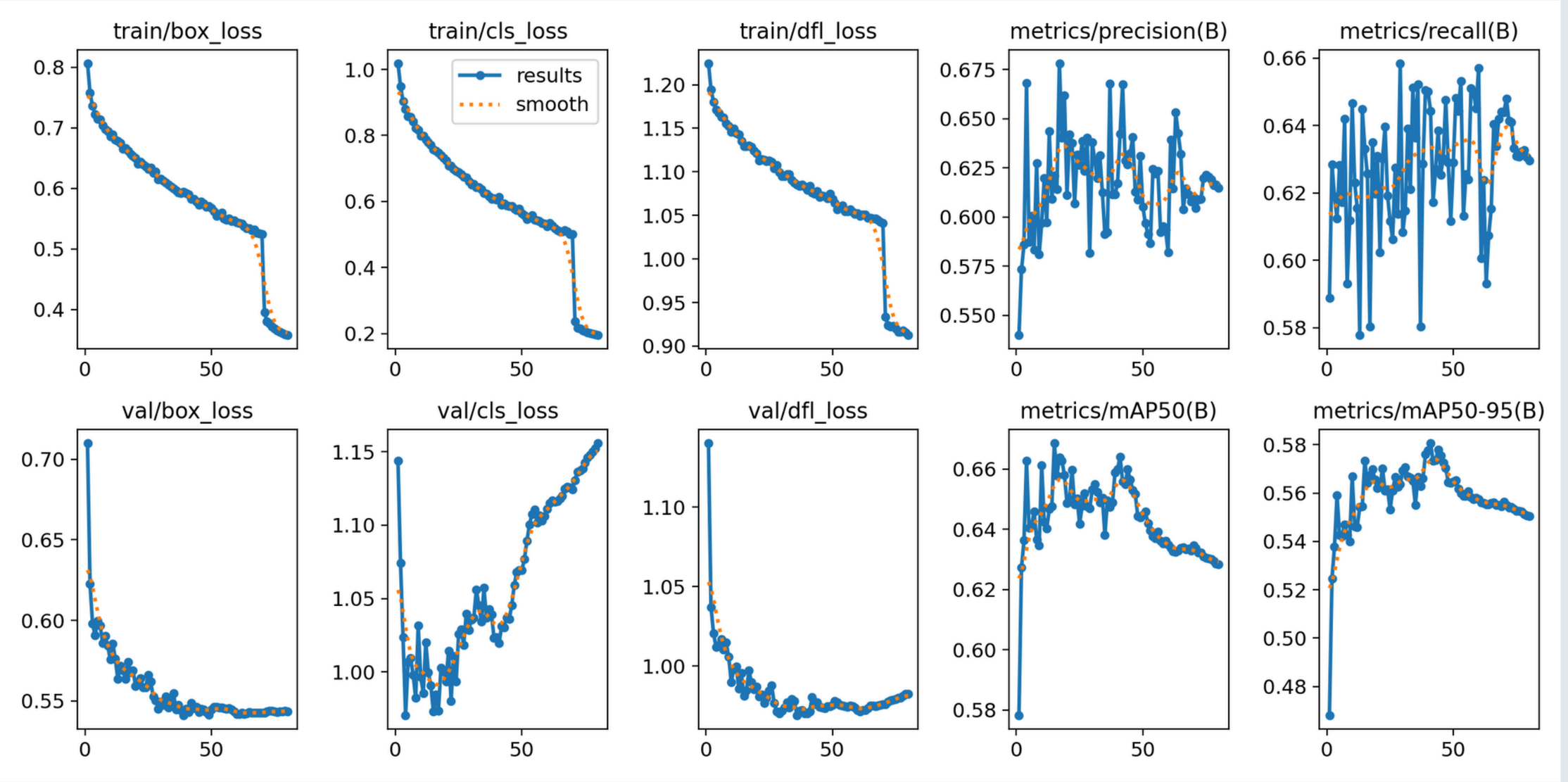
MÔ HÌNH

- Mô hình lựa chọn: YOLOv8 Medium (YOLOv8m)
- Môi trường huấn luyện: Lightning AI (GPU: NVIDIA Tesla T4)
- Cấu hình huấn luyện:

Tham số	Giá trị	Lý do
epochs	80	Đủ hội tụ, tránh overfit
imgz	640	Chuẩn YOLOv8, cân bằng tốc độ / chất lượng
optimizer	AdamW	Ổn định hơn SGD với dataset thời trang
lr0	0.0005	Nhỏ hơn default vì đang fine-tune
hsv_v	0.4	Thay đổi độ sáng → mô hình nhận diện được trang phục màu đen/tối
hsv_s, hsv_h	0.7, 0.015	Thay đổi độ bão hòa / màu sắc
.....	.....	.....

Object detection

Kết quả



Object detection

Kết quả

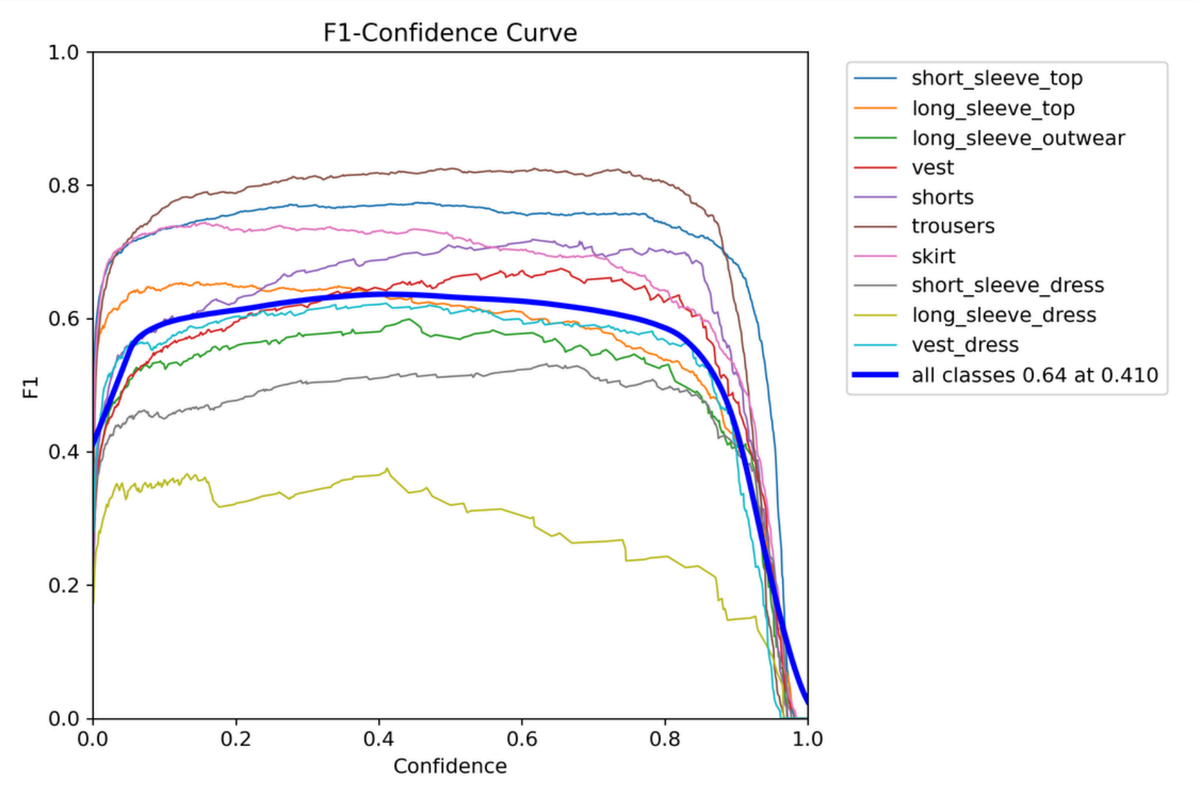
Mô hình đạt kết quả tốt nhất tại **epoch 41**:

Chỉ số	Giá trị
Precision	0.6423 (64.23%)
Recall	0.6444 (64.44%)
mAP@50	0.6640 (66.40%)
mAP@50-95	0.5807 (58.07%)

- **Precision (Độ chính xác)**: Trong tất cả những gì mô hình dự đoán là trang phục, bao nhiêu phần trăm là đúng thật sự?
- **Recall (Độ bao phủ)**: Trong tất cả những trang phục thực sự có trong ảnh, mô hình phát hiện được bao nhiêu phần trăm?
- **mAP@50**: Mean Average Precision tại ngưỡng IoU = 0.50. Đây là chỉ số tổng hợp đánh giá cả precision lẫn recall trên toàn bộ các lớp.
- **mAP@50-95**: Trung bình mAP tính trên nhiều ngưỡng IoU từ 0.50 đến 0.95 (nghiêm ngặt hơn, đánh giá chất lượng bounding box).

Object detection

Kết quả

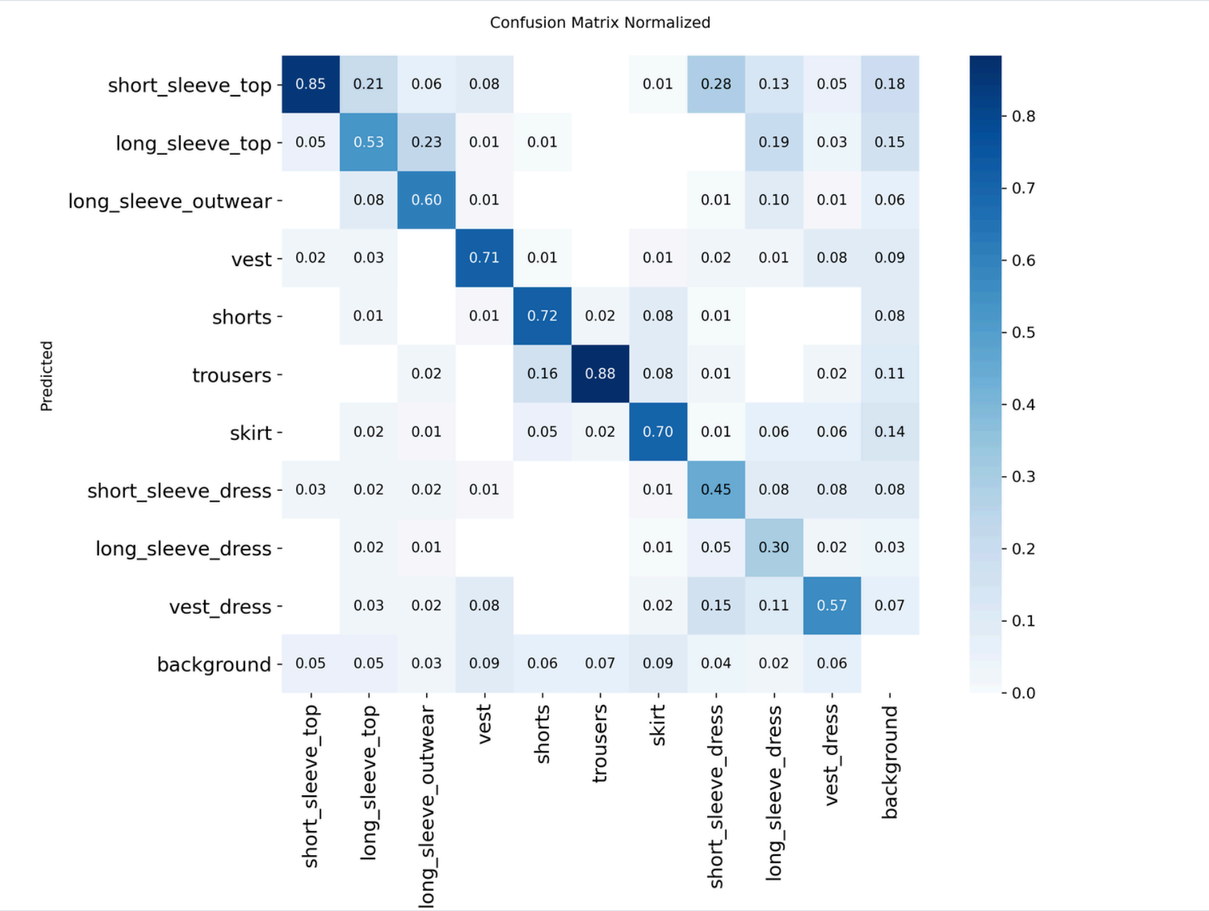


- F1 đạt đỉnh cao nhất ≈ 0.64 tại ngưỡng confidence ≈ 0.410
- Chọn confidence threshold = 0.5, ưu tiên Precision cao hơn

→ Chấp nhận bỏ sót một số trang phục nhưng đảm bảo những gì phát hiện được đều chính xác.

Object detection

Kết quả



- Đường chéo chính tương đối sáng
  → mô hình phân loại đúng là chủ yếu.
- background (cột cuối) thể hiện tỉ lệ false negative — trang phục có trong ảnh nhưng không phát hiện được

→ Mô hình đang phân biệt tốt các class có đặc trưng rõ ràng (shorts, skirt) và nhầm lẫn 1 số class có hình dạng tương đồng (short_sleeve_top, short_sleeve_dress)

Object detection

Kết quả





Embedding & Vector Search

- Khi user **upload outfit**:
- **YOLO detect** từng item
- hệ thống **crop từng item**
- mỗi crop được encode thành **vector query**
- PostgreSQL pgvector tìm sản phẩm gần nhất bằng **cosine distance**
- **distance** càng nhỏ → ảnh càng giống
- **Lấy 50 sản phẩm** gần nhát theo vector

Cosine Distance:

$$d_{cos}(x,y) = 1 - \frac{\sum_{i=0}^{n-1} x_i y_i}{\sqrt{\sum_{i=0}^{n-1} x_i^2} \times \sqrt{\sum_{i=0}^{n-1} y_i^2}}$$

Reranking

final_score =
  0.60 * image_similarity
+ 0.15 * category_score
+ 0.15 * color_score
+ 0.10 * text_style_score
- category_penalty
- color_penalty

image_similarity	Độ giống ảnh từ pgvector
category_score	Độ khớp loại trang phục
color_score	Độ khớp màu chính
text_style_score	Độ khớp mô tả/style
category_penalty	Phạt nếu sai nhóm trang phục
color_penalty	Phạt nếu lệch màu mạnh

Catogory score

Category score kiểm tra sản phẩm có đúng loại trang phục với class YOLO hay không, ví dụ áo thì ưu tiên shirt/top/t-shirt, quần thì ưu tiên pants/trousers/jeans.

- Cách tính:
  - Nếu product text chứa keyword category:
  - category_score = 0.55
  - Nếu subtype khớp: category_score+=0.15
- Giới hạn: 0.0 <= category_score <= 0.85

Catogory penalty

- Penalty giúp giảm điểm các sản phẩm sai loại rõ ràng. Hệ thống không loại bỏ cứng, chỉ trừ điểm để vẫn giữ tính linh hoạt khi YOLO detect chưa chắc chắn.
- Nếu query là áo nhưng product là pants/jeans/skirt:
  - category_penalty = 0.10
- Nếu không sai nhóm: category_penalty = 0.0

Color Score

- Vì crop quần áo thường có background hoặc da/người ở viền ảnh, hệ thống ưu tiên vùng trung tâm để lấy màu chính của trang phục.
  - center_ratio = 0.50
  - min_center_pixels = 32
- *Giảm ảnh hưởng của background sáng, tránh lấy nhầm màu nền thay vì màu quần áo.*
- *Hệ thống đo khoảng cách giữa hai màu trong không gian RGB, rồi chuyển thành điểm similarity.*
  - *distance_score =1 - RGB_distance / MAX_RGB_DISTANCE*
    - *distance_score gần 1 -> màu rất giống*
    - *distance_score gần 0 -> màu rất khác*

Color penalty

- Nếu màu lệch rõ ràng, hệ thống không chỉ cho color score thấp mà còn trừ thêm penalty để giảm khả năng sản phẩm sai màu xuất hiện ở top.
- Nếu màu khác nhau mạnh:
  - query_color_name != product_color_name
  - color_score < 0.35
- Thì bị trừ color_penalty = 0.08

Hạn chế

- Catalog còn nhỏ
- *Query của người dùng và ảnh sản phẩm trong database không cùng kiểu ảnh, nên embedding có thể chưa khớp hoàn toàn.*
- *Bbox chỉ cắt hình chữ nhật, chưa tách chính xác vùng quần áo*

KẾT LUẬN

Project chứng minh được một hệ thống AI/full-stack hoàn chỉnh cho bài toán fashion visual search: từ thu thập dữ liệu, lưu trữ ảnh, tạo embedding, tìm kiếm vector, reranking, đến giao diện người dùng và đánh giá kết quả.